

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Самарский национальный исследовательский
университет имени академика С.П. Королева»
(Самарский университет)

Институт естественнонаучный
Факультет механико-математический
Кафедра информатики и вычислительной математики

Направление подготовки
02.03.03 Математическое обеспечение и
администрирование информационных систем
Направленность (профиль) “Разработка и
администрирование информационных систем”

Курсовая работа по дисциплине “Базы данных и СУБД”

Разработка информационной системы учета повышения квалификации преподавателей ВУЗа

Выполнил студент
курса 3 группы 6345-020303D
Назарова Виктория Викторовна

Научный руководитель
к.ф-м.н., доцент
Русакова М.С.

Работа защищена
«_____» _____ 2022 г.
Оценка _____
Зав. кафедрой ИиВМ
д.ф.-м.н., профессор
Степанов А.Н.

Самара 2022

Содержание

Введение.....	4
1 Проектирование информационной системы учета повышения квалификации преподавателей ВУЗа.....	7
1.1 Описание предметной области.....	7
1.2 Обзор систем-аналогов.....	9
1.2.1 Информационная система «Повышение квалификации».....	9
1.3 Функциональные и нефункциональные требования к информационной системе.....	10
2 Проектирование и реализация базы данных учета повышения квалификации преподавателей ВУЗа.....	12
2.1 ER – диаграмма.....	12
2.2 Логическая модель базы данных.....	17
2.3 Нормализация реляционных отношений.....	17
2.4 Обоснование выбора СУБД.....	20
2.5 Физическая модель базы данных.....	21
3 Проектирование и реализация информационной системы учета повышения квалификации преподавателей ВУЗа.....	22
3.1 Клиент–серверная архитектура информационной системы.....	22
3.2 Диаграмма вариантов использования.....	23
3.3 Диаграммы последовательности действий.....	25
3.4 Выбор средств реализации.....	26

3.5 Реализация серверной части.....	27
3.6 Реализация клиентской части.....	30
3.7 Тестирование информационной системы.....	32
Заключение.....	42
Список используемых источников.....	43
ПРИЛОЖЕНИЕ А.....	45
ПРИЛОЖЕНИЕ Б.....	46
ПРИЛОЖЕНИЕ В.....	49
ПРИЛОЖЕНИЕ Г.....	55

Введение (большими буквами и по центру)

Информационные технологии в настоящее время широко применяются для автоматизации деятельности на различных предприятиях. Постоянно растет объем и сложность обрабатываемой информации, требуются новые виды ее представления. Организовать поиск, провести анализ и совершить многие другие операции значительно проще и удобнее, имея дело с электронными документами.

Объективно происходящий процесс информатизации общества ставит новые требования профессиональным качествам специалистов любого профиля в области применения средств ИКТ. В связи с этим, одним из приоритетных направлений информатизации современного общества является процесс информатизации образования, в центре которого является внедрение информационных технологий в предметных областях профессиональной деятельности педагогов и управляющей организации учебного процесса.

В настоящее время существует необходимость повышения профессионального уровня сотрудников, и слабое развитие информационных систем для образовательной отрасли делают актуальной задачу разработки информационной системы учета повышения квалификации преподавателей вуза.

Актуальность темы курсовой работы определена тем, что повышение квалификации преподаватели вузов должно проходить каждые три года, кроме того для переизбрания по конкурсу на замещение вакантной должности нужны и сведения о непросрочном повышении квалификации, отметка о которых должна быть в учебном отделе, поэтому необходимо вести учет повышения квалификации преподавателей вуза.

Повышение квалификации – это один из видов профессионального обучения работников предприятия, которое проводится с целью повышения

уровня теоретических знаний, совершенствования практических навыков и умений сотрудников организации в связи с постоянно повышающимися требованиями к их квалификации.

Целью курсовой работы является разработка информационной системы учета повышения квалификации преподавателей вуза, которая позволит вести учет повышения квалификации, учет планирования повышения квалификации сотрудников, контроль выполнения плана повышения квалификации, учет заявок на повышение квалификации.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области.
2. Сформулировать функциональные и нефункциональные требования к информационной системе.
3. Построить информационную модель данных (ОПК-4, ОПК-11).
4. Построить логическую модель данных (ОПК-4, ОПК-11).
5. Выбрать СУБД (ОПК-11).
6. Построить физическую модель данных (ОПК-4, ОПК-11).
7. Реализовать базу данных в выбранной СУБД (ОПК-4, ОПК-11).
8. Спроектировать приложение для работы с созданной базой данных (ОПК-4, ОПК-11).
9. Реализовать приложение с веб-интерфейсом, позволяющее вести учет повышения квалификации преподавателей вуза (ОПК-4, ОПК-11).
10. Провести тестирование созданной системы, оценить полученные результаты (ОПК-4, ОПК-9).

11. Разработать инструкцию пользователя для созданной системы (ОПК-9).

1 Проектирование информационной системы учета повышения квалификации преподавателей ВУЗа

1.1 Описание предметной области

Информационная система (ИС) — система, предназначенная для хранения, поиска и обработки информации, и соответствующие организационные ресурсы (человеческие, технические, финансовые и т. д.), которые обеспечивают и распространяют информацию.

Совершенствование и развитие направлений института и, прежде всего разработка, внедрение и освоение новых технологий требует систематического повышения профессионального уровня сотрудников и повышения квалификации в пределах конкретных трудовых условий. В повышении имеют заинтересованность и предприятия, и сами сотрудники, поскольку требования к качеству рабочей силы постоянно повышается.

Подготовка квалифицированных сотрудников представляет собой ряд мероприятий, которые направлены на систематическое получение и повышение квалификации, отвечающей текущим и перспективным целям института и обеспечивающей соответствие требований, предъявляемых начальством к способностям работника. Подготовка квалифицированного персонала может убрать необходимость привлечения работников со стороны и предотвратить его сокращение путем устранения дефицита рабочей силы. Информация о таких мероприятиях рассылается ежегодно на кафедры в виде объявлений.

В объявлении о повышении квалификации преподавателей указываются: название программы, руководитель программы, даты проведения, день недели, время и место проведения. Следует учесть, что преподавателей по некоторой программе может быть несколько: руководитель программы повышения квалификации и один или несколько преподавателей.

Преподаватели кафедры, у которых подошел срок очередного повышения квалификации, после ознакомления с планом данного мероприятия, должны оформить заявку. Данный документ формируется следующим образом: заведующий кафедрой указывает номер программы, по которой преподаватель хотел бы пройти курс повышения квалификации, данные направляемого преподавателя: ФИО, возраст, ученую степень, ученое звание, должность, педагогический стаж, контактный телефон и e-mail. После того, как заявка попадает в учебный отдел, формируется приказ, согласно которому преподаватели зачисляются в качестве слушателей на тот или иной курс повышения квалификации.

При поступлении на курс преподаватели заполняют учебную карточку слушателя, в которой нужно указывать следующее: наименование дополнительной образовательной программы, сроки ее обучения, фамилия, имя и отчество слушателя, дата рождения, образование (какой ВУЗ, год окончания и квалификация по диплому), место работы и занимаемая должность (ВУЗ, кафедра, должность, преподаваемая дисциплина), адрес постоянного места жительства, телефон, e-mail, повышение квалификации в предыдущие годы (год, образовательное учреждение, форма повышения квалификации, наименование дополнительной образовательной программы).

Преподаватели, прошедшие курс повышения квалификации, должны выполнить выпускную квалификационную работу (либо пройти квалификационное тестирование). У данной работы есть научный руководитель, который может быть руководителем курса повышения квалификации или одним из преподавателей этого курса, также тема и дата защиты. После защиты ВКР выставляется оценка.

По окончании курсов повышения квалификации слушателям, успешно прошедшим защиту ВКР (или итоговое тестирование), выдается свидетельство о повышении квалификации государственного образца. В нем указываются: полное официальное название учебного заведения;

регистрационные номер, под которым документ внесен во внутреннюю книгу регистрации; ФИО выпускника курсов; название и объем программы в академических часах; период обучения; наименование дисциплин программы, по которым сданы зачеты и экзамены за время обучения (если таковое предусмотрено программой), количество часов их проведения и оценка; тема итоговой работы; дата выдачи сертификата.

1.2 Обзор систем-аналогов

1.2.1 Информационная система «Повышение квалификации»

Web-приложение «Повышение квалификации» предназначено для учёта и управления отчётными документами, связанными с повышением квалификации профессорско-преподавательского состава кафедры «Информационные системы» Северо-Казахстанского государственного университета имени М. Козыбаева. Web-приложение позволяет проинформировать пользователей о квалификациях преподавателей, а также внести и редактировать информацию о квалификации преподавателей (Рисунок 1) [1].

Повышение квалификации

Нормативные документы

Запланированные

Пройденные

Запланированные квалификации

Выберите год:

Выберите диапазон:

№	ФИО	Сроки прохождения	Место прохождения	Тематика	Дата проведения курсов	Примечания	Редактировать	Удалить
1	Долматова Лариса Владимировна	40	ИППК СКГУ	Изучение государственного языка	2012-04-03	ИППК СКГУ при наличии группы(за счет СКГУ)		
2	Шевчук Ольга Владимировна	30	ИППК СКГУ	Методика дистанционного обучения	2011-04-06	ИППК СКГУ, при наличии группы(за счет СКГУ)		
3	Ушакова Екатерина Вячеславовна	40	ИППК СКГУ	Изучение государственного языка	2011-02-08	ИППК СКГУ, при наличии группы(за счет СКГУ)		

Рисунок 1 - Страница модуля «Запланированные квалификации»

1.3 Функциональные и нефункциональные требования к информационной системе

Необходимо разработать информационную систему учета повышения квалификации преподавателей вуза. У информационной системы существует 3 уровня доступа: администратор, преподаватель, гость. Основные функции, которые должна поддерживать разрабатываемая информационная система:

- 1) хранение и отображение информации о пройденных и запланированных повышениях квалификации для конкретного преподавателя;
- 2) возможность для администратора и преподавателя проводить редактирование или удаление внесенной информации; просмотр информации

о пройденных и запланированных повышениях квалификации других преподавателей;

3) возможность для администратора добавления информации о пройденном или запланированном повышении квалификации; добавление и редактирование программы повышения квалификации;

4) возможность для преподавателя добавления, редактирования и удаления личной информации о пройденном или запланированном повышении квалификации; просмотра программы повышения квалификации и записи на нее; заполнения учебной карточки слушателя;

5) возможность для гостя просмотра информации о пройденных и запланированных повышениях квалификации;

6) формирование отчётности о прошедшем и запланированном повышении квалификации преподавателей.

Информационная система должна соответствовать следующим критериям:

- 1) интуитивно понятный интерфейс;
- 2) приемлемая скорость отклика сервера на действия пользователей;
- 3) система должна функционировать в ОС семейства Windows;
- 4) разграничение прав доступа у администратора, преподавателя и гостя;
- 5) система должна выдерживать одномоментное обращение более 300 пользователей.

Были сформированы требования к тестированию:

- 1) тестирование работы системы в браузерах Chrome, Яндекс, MS Edge;

- 2) провести нагрузочное тестирование системы, в котором имитировать запросы 500 пользователей в течение 4-5 секунд;
- 3) функциональное тестирование.

2 Проектирование и реализация базы данных учета повышения квалификации преподавателей ВУЗа

2.1 ER – диаграмма

Для создания ER-диаграммы проектируемой системы учёта повышения квалификации преподавателей ВУЗа на основе сформулированной предметной области, используем нотацию IDEF1X. IDEF1X является методом для разработки реляционных баз данных и использует условный синтаксис, разработанный для удобного построения концептуальной схемы. Средства моделирования IDEF1X специально разработаны для построения реляционных информационных систем. Сущность описывается в диаграмме IDEF1X графическим объектом в виде прямоугольника. Верхняя часть называется ключевой областью, а нижняя часть областью данных. В IDEF1X концепция зависимых и независимых сущностей усиливается типом взаимосвязей между двумя сущностями. Для передачи внешнего ключа в дочернюю сущность создаётся идентифицирующая связь между родительской и дочерней сущностью. Идентифицирующие взаимосвязи обозначаются сплошной линией между сущностями. Неидентифицирующие связи, являющиеся уникальными для IDEF1X, также связывают родительскую сущность с дочерней, однако используются для отображения другого типа передачи атрибутов внешних ключей – передача в область данных дочерней сущности (под линией). Неидентифицирующие связи отображаются пунктирной линией между объектами. Так как переданные ключи в неидентифицирующей связи не являются составной частью первичного ключа дочерней сущности, то этот вид связи не проявляется ни в одной идентифицирующей зависимости [3].

Определим сущности и атрибуты сущностей для предметной области учета повышения квалификации преподавателей вуза.

Сущность «Преподаватель» определяет человека, который желает пройти курс повышения квалификации. Атрибуты сущности «Преподаватель»:

- имя;
- фамилия;
- отчество;
- дата рождения;
- должность;
- ученая степень;
- ученое звание;
- стаж;
- кафедра;
- электронная почта;
- телефон.

Сущность «Учебная карточка слушателя» определяет документ, который заполняется преподавателем при поступлении на курс повышения квалификации. Атрибуты сущности «Учебная карточка слушателя»:

- образование слушателя;
- место работы;
- адрес постоянного места жительства.

Сущность «Программа повышения квалификации» определяет обучающие курсы, по итогам прохождения которых преподавателю выдается

свидетельство государственного образца по итогам повышения квалификации. Атрибуты сущности «Программа повышения квалификации»:

- название;
- руководитель;
- дата проведения;
- длительность;
- время проведения;
- место проведения;
- преподаватель программы.

Сущность «Выпускная квалификационная работа» определяет работу, которая выполняется преподавателем после прохождения курса повышения квалификации. Атрибуты сущности «Выпускная квалификационная работа»:

- научный руководитель;
- тема;
- дата защиты;
- оценка.

Сущность «Свидетельство о повышении квалификации» определяет документ, который подтверждает прохождение курса повышения квалификации. Атрибуты сущности «Свидетельство о повышении квалификации»:

- регистрационный номер;
- название учебного заведения;
- дата выдачи.

Сущность «Повышение квалификации в предыдущие годы» определяет перечень программ, по которым преподаватель ранее проходил курсы повышения квалификации. Атрибуты сущности «Повышение квалификации в предыдущие годы»:

- год прохождения;
- образовательное учреждение;
- форма повышения квалификации;
- наименование программы.

Сущности предметной области учета повышения квалификации преподавателей вуза связаны следующим образом:

- сущности «Преподаватель» и «Программа повышения квалификации» связаны между собой связью «многие ко многим»: один преподаватель может быть записан на несколько программ повышения квалификации, на одну программу повышения квалификации может быть записано несколько преподавателей;

- сущности «Преподаватель» и «Учебная карточка слушателя» связаны между собой связью «один ко многим»: один преподаватель может иметь несколько карточек слушателя, в одной карточке слушателя может быть только один преподаватель;

- сущности «Преподаватель» и «Выпускная квалификационная работа» связаны между собой связью «один ко многим»: один преподаватель может защищать несколько выпускных квалификационных работ, и выпускная квалификационная работа может быть защищена только одним преподавателем;

- сущности «Преподаватель» и «Свидетельство о повышении квалификации» связаны между собой связью «один ко многим»: один преподаватель может получить несколько свидетельств о повышении квалификации, одно свидетельство может быть получено только одним преподавателем;

- сущности «Программа повышения квалификации» и «Учебная карточка слушателя» связаны между собой связью «один ко многим»: на одной программе может быть несколько учебных карточек слушателей, в

одной карточке слушателя может быть записана только одна программа повышения квалификации;

– сущности «Программа повышения квалификации» и «Свидетельство о повышении квалификации» связаны между собой связью «один ко многим»: на одной программе повышения квалификации может быть выдано несколько свидетельств о повышении квалификации, в одном свидетельстве о повышении квалификации может быть записана одна программа повышения квалификации;

– сущности «Учебная карточка слушателя» и «Повышение квалификации в предыдущие годы» связаны между собой связью «один ко многим»: в одной учебной карточке слушателя может быть несколько записей о повышении квалификации в предыдущие годы, одна запись о повышении квалификации в предыдущие годы может быть указана только в одной карточке слушателя.

На рисунке 2 показана созданная ER-диаграмма проектируемой системы учёта повышения квалификации преподавателей ВУЗа.



Рисунок 2 - ER-диаграмма системы учёта повышения квалификации преподавателей ВУЗа

2.2 Логическая модель базы данных

Построим логическую модель базы данных системы учета повышения квалификации преподавателей вуза, опираясь на ER-диаграмму. В ПРИЛОЖЕНИИ А показана построенная логическая модель базы данных проектируемой системы. Созданная логическая модель базы данных содержит следующие отношения: «Слушатель», «Преподаватель», «Заявка на повышение квалификации», «Программа повышения квалификации», «Свидетельство о повышении квалификации», «Учебная карточка слушателя», «Выпускная квалификационная работа», «Повышение квалификации в предыдущие годы», «Образовательное учреждение», «Ученая степень», «Ученое звание», «Кафедра». Для разбиения связи многие-ко-многим было введено промежуточное отношение: «Заявка на повышение квалификации». Все отношения имеют связи «один ко многим». Все отношения логической модели базы данных находятся в третьей нормальной форме, поскольку каждое отношение модели находится во второй нормальной форме (каждый неключевой атрибут каждого отношения имеет неприводимую функциональную зависимость от соответствующего первичного ключа) и каждый неключевой атрибут каждого отношения не находится в транзитивной функциональной зависимости от соответствующего первичного ключа.

2.3 Нормализация реляционных отношений

Метод семантического моделирования, который использовался при построении логической модели данных, состоит в осмыслении искомой предметной области. Получившаяся в результате этого логическая модель, является слабо формализованной, из-за чего проектируемая база данных, может быть, некорректной и неоптимальной вследствие избыточности данных [2].

При работе с отношениями, в которых присутствует избыточность данных, могут возникнуть аномалии:

- аномалия обновления;
- аномалия включения;
- аномалия удаления.

Для устранения аномалий, следует воспользоваться теорией нормализации. Теория нормализации, в отличие от метода семантического моделирования, является строго формализованной [2].

В теории нормализации все реляционные отношения разбиваются на категории, и каждая такая категория объявляется нормальной формой.

Процесс проектирования БД с использованием метода нормализации заключается в том, что исходное отношение путем декомпозиции переводят из младшей нормальной формы в более высокую нормальную форму. Если база данных находится хотя бы в третьей нормальной форме, ее можно считать нормализованной.

Для примера, рассмотрим отношение «Слушатель» (Рисунок 3).

Слушатель	
РК	<u>ID слушателя</u>
	Преподаватель
	Кафедра

Рисунок 3 – Отношение «Слушатель»

Рассматриваемое отношение не имеет нормализации, так как не находится даже в 1 НФ. Атрибут «Преподаватель» не является атомарным, его можно разделить на более простые атрибуты, поэтому она не отражает затрагиваемую в курсовой предметную область. Для приведения данного отношения в 1НФ необходимо атрибут «Преподаватель» разбить на десять – «Имя», «Фамилия», «Отчество», «Дата рождения», «Должность», «Ученая

степень», «Ученое звание», «Стаж», «Контактный телефон», «Email» (Рисунок 4).

Слушатель	
РК	ID слушателя
	Имя
	Фамилия
	Отчество
	Дата рождения
	Должность
	Ученая степень
	Ученое звание
	Стаж
	Контактный телефон
	Email
	Кафедра

Рисунок 4 – Отношение «Слушатель» в 1 НФ

Теперь отношение «Слушатель» находится в 1НФ. Теперь рассмотрим отношение нахождение в 2НФ. Отношение находится во 2НФ, если оно находится в 1НФ и каждый неключевой атрибут неприводимо зависит от ПК. Из определения 2НФ, понятно, что отношение «Слушатель» в нем не находится. Для приведения данного отношения к 2НФ необходимо воспользоваться метод декомпозиции. Отношение «Слушатель» можно декомпозировать на четыре отношения «Слушатель», «Ученая степень», «Ученое звание», «Кафедра» то есть выделить атрибуты «Ученая степень», «Ученое звание», «Кафедра», как отдельные отношения и связать их с отношением «Слушатель» (Рисунок 5).

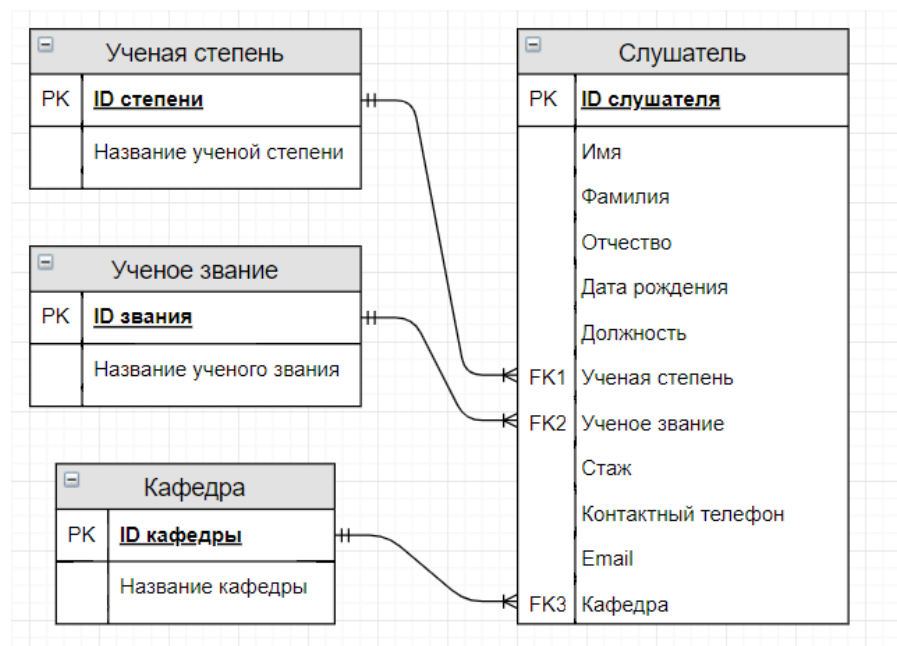


Рисунок 5 – Отношение «Слушатель» в 2 НФ

Таким образом условие 2НФ было выполнено. Теперь нужно проверить условие 3НФ. Отношение находится в 3НФ, когда находится в 2НФ и каждый не ключевой атрибут функционально зависит от первичного ключа. После декомпозиции отношения «Слушатель» находится в 3НФ, следовательно, отношение нормализовано.

Все отношения, находящихся в базе данных, также были нормализованы до 3НФ.

2.4 Обоснование выбора СУБД

Для реализации проектируемой системы использована СУБД MySQL - свободная объектно-реляционная система управления базами данных. MySQL работает на ОС семейств Windows, Linux, Unix, Solaris и других.

Достоинствами MySQL является то, что она использует многопоточность, поддержку нескольких одновременных запросов, поддерживает разнообразные типы данных, использует гибкую систему привилегий и паролей, поддерживает быструю работу, масштабируемость,

содержит интерфейсы API для большинства языков программирования: C, C++, PHP, Python, Ruby, Java и другие.

2.5 Физическая модель базы данных

Построим физическую модель базы данных проектируемой системы для СУБД MySQL. Построенная модель показана на рисунке 6.

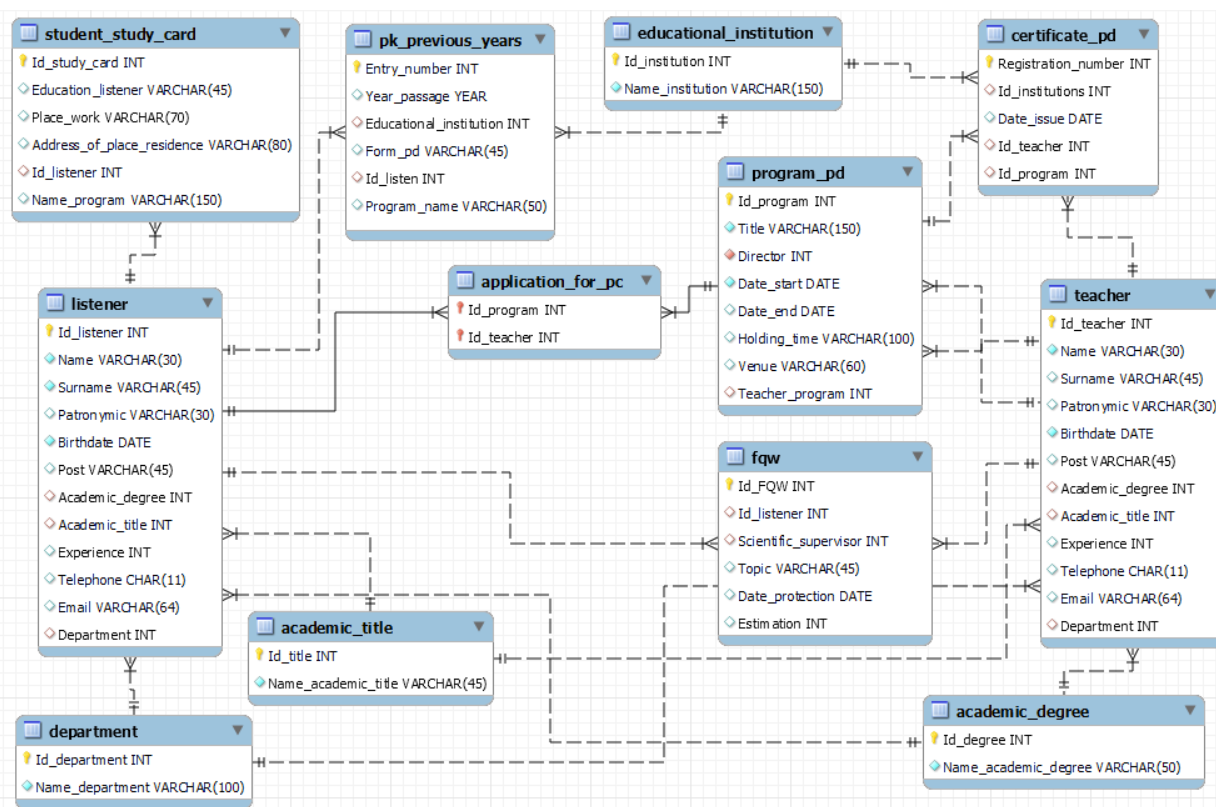


Рисунок 6 – Физическая модель базы данных системы учета повышения квалификации преподавателей ВУЗа

3 Проектирование и реализация информационной системы учета повышения квалификации преподавателей ВУЗа

3.1 Клиент–серверная архитектура информационной системы

Разрабатываемая информационная система, является веб–приложением. Веб–приложение – это клиент–серверное приложение, в котором клиент (веб–интерфейс) запускается в веб–браузере, а серверная часть запускается на веб–сервере [4].

Клиент–серверная архитектура веб–приложения будет трехуровневой. Она подразумевает сетевую или вычислительную архитектуру, в которой алгоритмы, вычислительная и сетевая нагрузки распределены между серверами и клиентами.

Достоинствами такой архитектур является:

- масштабируемость;
- высокая безопасность;
- центральное хранилище файлов;
- высокая надёжность;
- низкие требования к скорости канала (сети) между терминалами и сервером приложений [5].

Трехуровневая архитектура показана на рисунке 7.

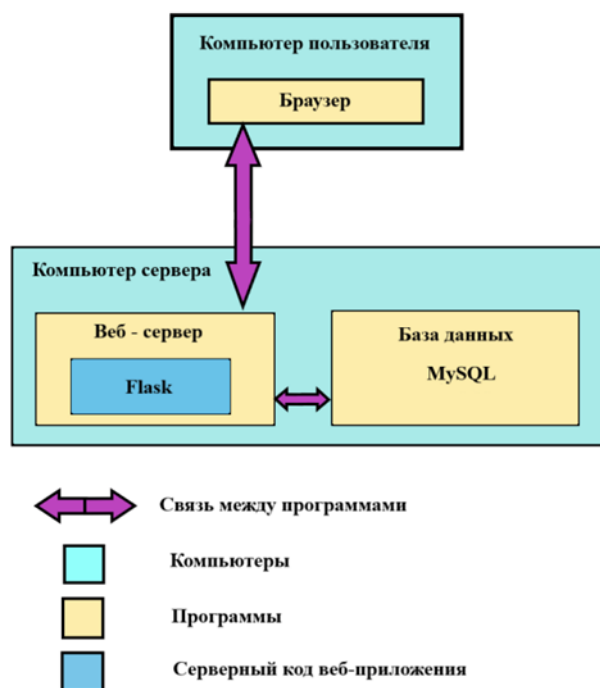


Рисунок 7 – Трехуровневая архитектура клиент–сервер

3.2 Диаграмма вариантов использования

Диаграмма вариантов использования (Use case diagram, диаграмма прецедентов) – диаграмма, на которой отражены отношения, существующие между актёрами (actors) и вариантами использования системы (ВИС). Диаграмма прецедентов предоставляет возможность заказчику, конечному пользователю и разработчику совместно обсуждать поведение системы и функциональность. Диаграмма вариантов использования позволит:

- разработать начальную концептуальную модель системы для её последующей детализации в форме логических и физических моделей;
- определить общие границы функциональности проектируемой системы;
- специфицировать требования к функциям проектируемой системы в форме вариантов использования;
- подготовить исходную документацию для взаимодействия разработчика, заказчика и конечного пользователя.

На рисунке 8 показана диаграмма вариантов использования системы учёта повышения квалификации преподавателей вуза.

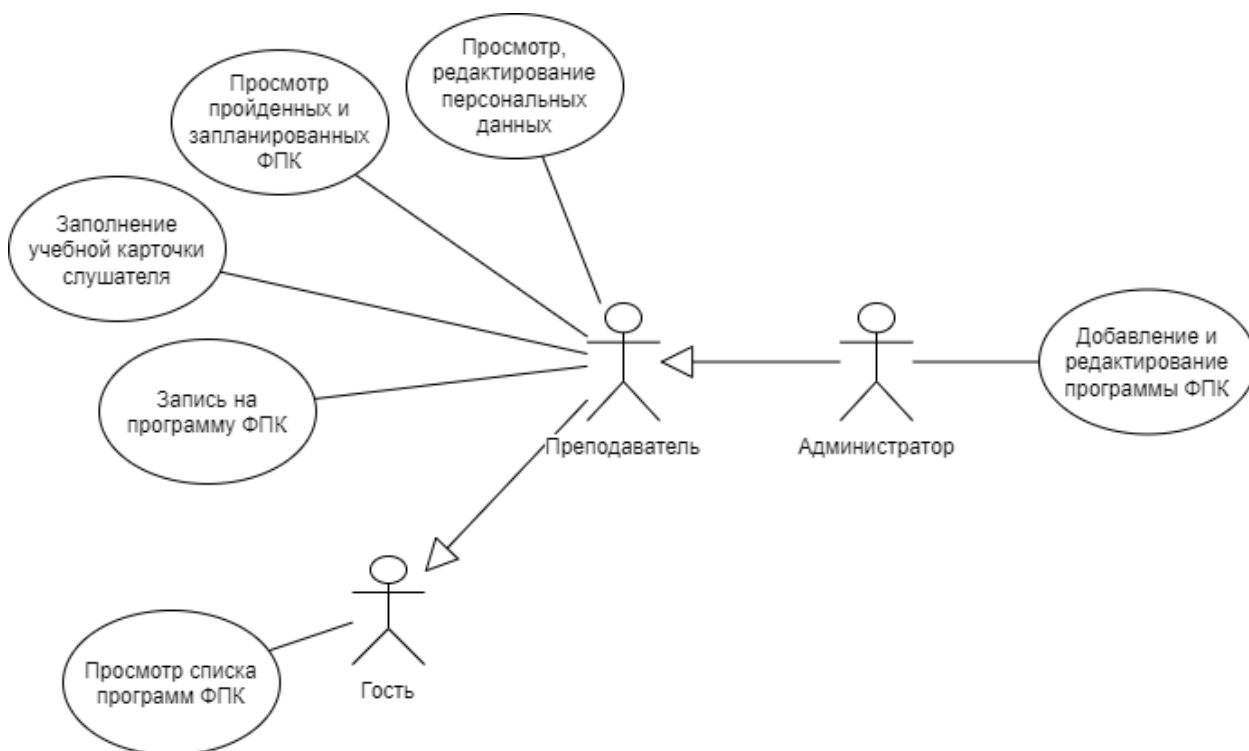


Рисунок 8 – Диаграмма вариантов использования информационной системы учета повышения квалификации преподавателей ВУЗа

Проектируемую систему могут использовать 3 различных актёра: «Гость», «Преподаватель», «Администратор». Актёру «Гость» доступен вариант использования «Просмотр программы повышения квалификации». Актёру «Преподаватель» доступны варианты использования «Просмотр, редактирование персональных данных», «Просмотр пройденных и запланированных курсов повышения квалификации», «Заполнение учебной карточки слушателя», «Запись на программу повышения квалификации», к тому же он наследует вариант использования актёра «Гость». Актёру «Администратор» доступен вариант использования «Добавление и редактирование программы повышения квалификации», так же он наследует все варианты использования актёра «Преподаватель».

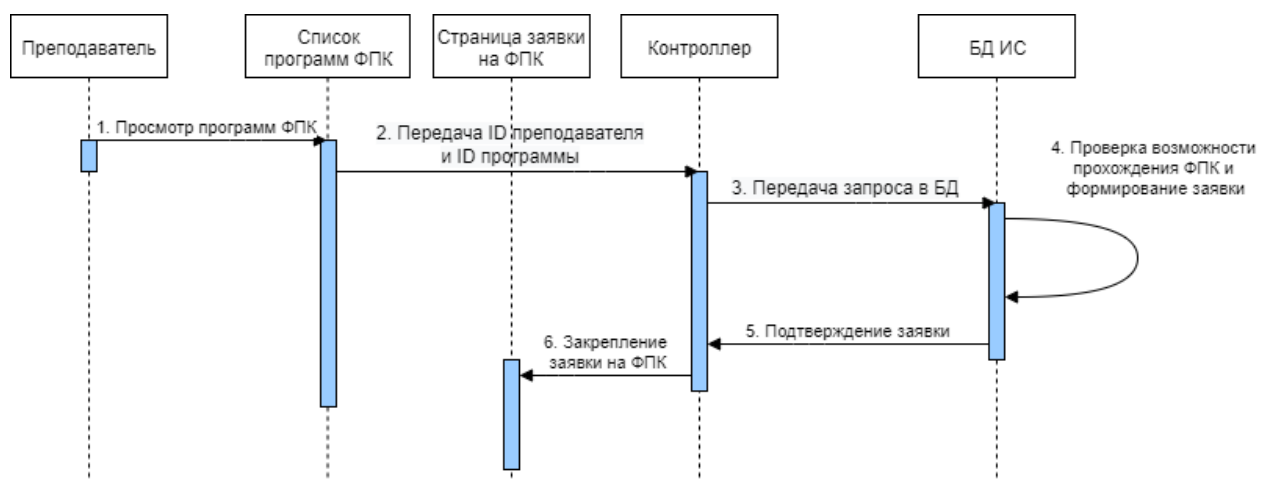
3.3 Диаграммы последовательности действий

Диаграмма последовательности (англ. Sequence diagram) относится к диаграммам взаимодействия UML, которые используются для моделирования динамических аспектов систем. Диаграмма последовательности подчёркивает временной порядок сообщений и изображается как таблица, в которой представлены объекты, расположенные вдоль оси X, и сообщения, упорядоченные по ходу времени – вдоль оси Y.

Диаграмма последовательности позволят описать поведение системы на уровне отдельных объектов, которые обмениваются между собой сообщениями, чтобы достичь нужной цели или реализовать некоторый вариант использования.

На рисунке 9 изображена диаграмма последовательности для варианта использования

«Заполнение зая



вки на ПК». (почему-то тут произошел разрыв слова, надо бы поправить)

Рисунок 9 - Диаграмма последовательности для варианта использования
«Заполнение заявки на ПК»

На рисунке 10 изображена диаграмма последовательности для варианта использования «Заполнение учебной карточки слушателя».

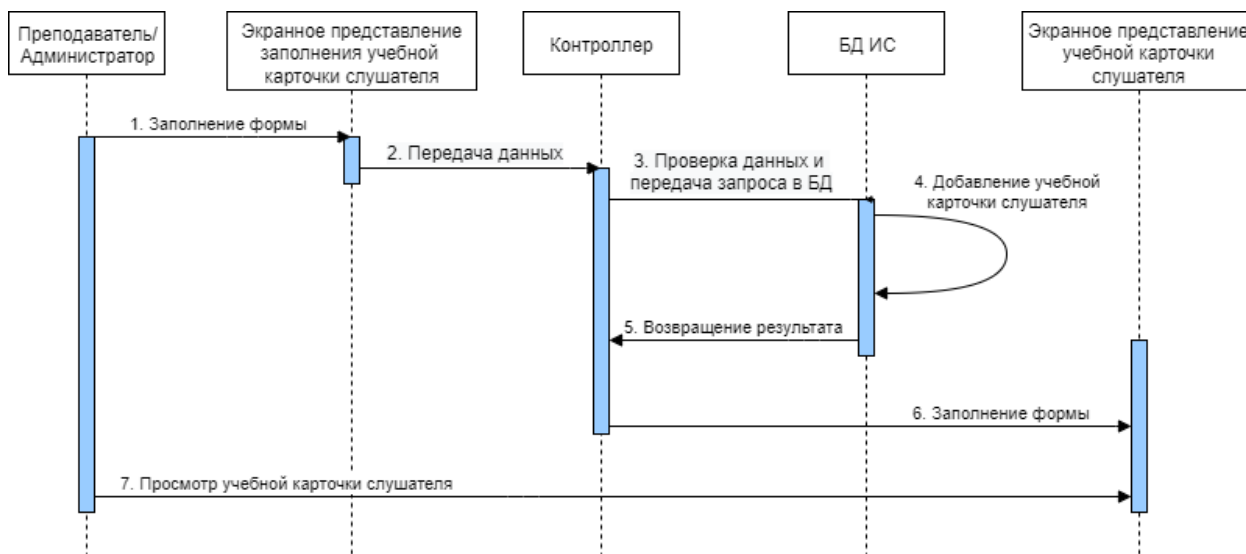


Рисунок 10 - Диаграмма последовательности для варианта использования «Заполнение учебной карточки слушателя»

3.4 Выбор средств реализации

Для разработки информационной системы учета повышения квалификации преподавателей ВУЗа использованы язык программирования Python, фреймворк Flask, шаблонизатор jinja, язык гипертекстовой разметки HTML и язык описания внешнего вида CSS. База данных для проектируемой информационной системы реализована в СУБД MySQL. В качестве среды разработки проектируемой системы и создания локального сервера используется PyCharm.

Python – высокоуровневый язык программирования общего назначения с динамической строгой типизацией и автоматическим управлением памятью, ориентированный на повышение производительности разработчика, читаемости кода и его качества, а также на обеспечение переносимости написанных на нём программ. Язык является полностью объектно–ориентированным – всё является объектами [8].

Flask – фреймворк для создания веб–приложений на языке программирования Python. Относится к категории так называемых микрофреймворков — минималистичных каркасов веб–приложений, сознательно предоставляющих лишь самые базовые возможности [7].

Jinja – это шаблонизатор для языка программирования Python. Он предоставляет Python–подобные выражения, позволяет настраивать теги, фильтры, тесты и глобальные переменные. Это текстовый шаблонизатор, поэтому он может быть использован для создания любого вида разметки, а также исходного кода [9].

MySQL – это кроссплатформенная БД, которая поддерживает достаточно полный набор команд SQL и доступна в исходных кодах. MySQL хранит всю базу данных (включая определения, таблицы, индексы и данные) в единственном стандартном файле на том компьютере, на котором выполняется программа [6].

3.5 Реализация серверной части

Серверная часть – это программа или скрипт на веб–сервере, обрабатывающая запросы браузера. При каждом действии пользователя на веб–странице, браузер отправляет запрос на веб–сервер, он, в свою очередь, обрабатывает этот запрос, вызывая flask–скрипт, который в зависимости от того, к чему формируется запрос, либо формирует веб–страничку, либо обращается к базе данных. Обратившись к ней, он извлекает из нее данные, которые необходимы для формирования страницы, запрошенной пользователем.

В серверной части были реализованы следующие основные функции на стороне сервера: `@app.route` – нужен для связывания url–адреса с функцией. Функции, которые объявлены после объявления `@app.route` с каким либо url,

содержат в себе различные методы. В конце каждой функции есть метод `return render_template`, который служит для отрисовки шаблонов и передачи в шаблоны некоторых данных.

В этих функциях применяются следующие методы:

- `cur.execute('SELECT * FROM название таблицы ').fetchall()` служит для вывода данных таблиц из БД;
- `request.form[""]` нужен для принятия данных, которые передал шаблон;
- `return redirect (url_for())` служит для перенаправления пользователя из одного шаблона в другой;
- `flash()` создает мгновенное сообщение вверху шаблона.

С помощью метода `myconn=pymysql.connect()` происходит подключение к БД MySQL (здесь требуется указать четыре параметра: имя хоста; имя пользователя; пароль; название базы данных) и файлу БД. Из объекта подключения `con` создается курсор `cur = myconn.cursor()`. Курсор используется для перебора кортежей базы данных.

Для понимания работы серверной части веб–приложения рассмотрим некоторые функции. В функции добавления новой программы повышения квалификации, используются следующие методы (Листинг Б.1):

```
- sql = "SELECT Id_teacher, Concat(Surname, ' ', Name, ' ', Patronymic)
FROM teacher"
```

```
cur.execute(sql)
```

```
direct = cur.fetchall()
```

производим перебор кортежей таблицы «Преподаватель»;

```
- title = request.form['title'] – request.form забирает из поля ввода формы
html введенные пользователем данные;
```

- sql = "SELECT Id_teacher FROM teacher WHERE Surname LIKE '%s'"

```
% director.split()[0]
```

```
cur.execute(sql)
```

```
id_direct = str(cur.fetchall()[0][0])
```

производим поиск первичного ключа таблицы «Преподаватель» по атрибуту Surname;

```
- cur.execute("INSERT INTO program_pd (Title, Director, Date_start,
Date_end, Holding_time, Venue, Teacher_program) VALUES (%s, %s, %s, %s,
%s, %s, %s)", (title, id_direct, date_start, date_end, holding_time, venue,
id_teach))
```

производим добавление нового кортежа в базу данных

- myconn.commit() – применяет изменения, внесенные в базу данных.

В функции входа в учетную запись, используются следующие методы (Листинг Б.2):

- cur.execute("SELECT * FROM user WHERE email=%s", (email)) – поиск кортежа с логином, введенным в поле email, таблицы «пользователи»;

- session['email'] = account[1] – создание сессии пользователя с соответствующим email;

- msg = 'Некорректный логин / пароль!' – сообщение, выводимое при неправильно (слово пишется слитно) введенном логине и пароле.

3.6 Реализация клиентской части

Клиентская часть разрабатываемого веб–приложения включает код HTML, CSS и Python (Flask), который открывается в браузере для отображения веб–страниц, составляющих интерфейс приложения, и управления их поведением при взаимодействии пользователя со страницей.

Последним уровнем разрабатываемой архитектуры будет клиентская часть веб–приложения. Она будет иметь структуру динамических веб–страниц, так как имеет значительные плюсы по сравнению со статическими:

- широкая функциональность сайта;
- простота обновления данных;
- легкая поддержка.

В результате проектирования было создано веб–приложение, разработка которого была разбита на этапы. Для начала были созданы макеты страниц, написанные на языке разметки HTML. При разработке нужно было учесть такие особенности, как ссылочная структура сайта, динамическое отображение информации о программах повышения квалификации и преподавателях. Все эти возможности реализованы с помощью flask и шаблонизатора jinja. В результате были реализованы следующие веб–страницы:

– Программы повышения квалификации – страница, на которой находится краткая информация о каждой программе повышения квалификации (Рисунок В.1).

– Информация о преподавателе – страница, на которой находится поиск преподавателя. Когда пользователь заполняет его и нажимает кнопку «Поиск», происходит обновление страницы и на ней появляются результаты поиска преподавателя (Рисунок В.2).

- Запись на программу повышения квалификации – страница, на которой находится форма для заполнения заявки на повышение квалификации (Рисунок В.3).

- Вход – страница, где пользователь вводит свой логин и пароль, и после нажатия кнопки «Войти», попадает на главную страницу (Рисунок В.4).

- Страницы подробной информации о программах, преподавателях – страницы, на которых отображается полная информация о каждой программе повышения квалификации или о каждом преподавателе, которая динамически формируется в зависимости от того, на ссылку какой программы пользователь нажал на главной странице или фамилию какого преподавателя ввел на странице поиска (Рисунок В.5, Рисунок В.6).

- Различные страницы редактирования информации о программах, преподавателях (Рисунок В.7, Рисунок В.8).

Для понимания работы клиентской части веб–приложения рассмотрим некоторые функции. В html шаблоне добавления новой программы повышения квалификации, используются следующие методы (Листинг Б.3):

- `<form method="post">` – сообщает серверу о методе запроса post. Метод post посылает на сервер данные в запросе браузера;

- `<datalist id="list">` – создает выпадающий список, позволяющий пользователю выбрать какой–либо пункт этого списка. Список связывается с текстовым полем с помощью атрибута id;

- `{% for i in content %} {% endfor %}` – цикл jinja, с помощью которого происходит перебор всех атрибутов таблицы «Программа повышения квалификации»;

- `"{{ request.form['title'] }}"` – извлекает значение из формы и передает его на сервер.

В html-шаблоне входа в учетную запись, используются следующие методы (Листинг Б.4):

- `<input id="email" name="email" type="text" class="textbox"/>` – образует текстовое поле ввода данных, и возвращает его в `request.form['email']` на сервере, после нажатия кнопки «Войти»;
- `<input type="submit" class="btn btn-warning" value="Войти">` – элемент класса `button` представляет из себя кнопку, посылает все введенные в поля ввода данные на сервер;
- `<div class="msg">{{ msg }}</div>` – является сообщением, которое отображается при неверном вводе логина и пароля, посылаемое сервером.

3.7 Тестирование информационной системы

Тестирование проводилось с помощью программы для проведения нагрузочного тестирования `jmeter`. Тестирование проводилось с целью узнать сколько пользователей одновременно может заходить на главную страницу сайта и авторизоваться на сайте. Тесты проводились со следующими параметрами: 500 виртуальных пользователей в течение 1 секунды делали 1 запрос к главной странице сайта или 1 запрос на авторизацию.

Параметры изображены на рисунке 11.



Thread Properties	
Number of Threads (users):	500
Ramp-up period (seconds):	1
Loop Count:	☐ Infinite 1

Рисунок 11 – Параметры нагрузочного тестирования

В ходе теста, с целью узнать сколько пользователей одновременно может заходить на главную страницу сайта, были получены следующие результаты, изображенные на рисунке 12.

- Sample– количество отправленных запросов равно 500;
- Average– Среднее арифметическое для всех ответов (сумма всех раз /число) равно 844 (каких единиц? Это среднее время отклика?);
- Min (ms) –Минимальное время отклика (мс) равно 43 мс;
- Max (ms) – Максимальное время отклика (мс) равно 1706 мс;
- Std Dev — Стандартное отклонение равно 383,69;
- Errorrate – процент неудачных тестов равно 0 процентов;
- Throughput – сколько запросов в секунду обрабатывает сервер равно 253,2 запроса в секунду;
- Received KB/Sec - количество полученных данных (кб/сек) равно 6135,28 кб/сек;
- Sent KB/Sec – количество отправленных данных (кб/сек) равно 29,42 кб/сек;
- Avg. Bytes – средний размер ответа равно 24816 байт.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	500	844	43	1706	383.69	0.00%	253.2/sec	6135.28	29.42	24816.0
TOTAL	500	844	43	1706	383.69	0.00%	253.2/sec	6135.28	29.42	24816.0

Рисунок 12 – Результаты нагрузочного тестирования

Из результатов можно увидеть, что количество отправленных запросов равно 500, а процент ошибок составляет 0 процентов, из этого можно сделать вывод о том, что система выдерживает одновременное обращение 500 пользователей.

В ходе теста, с целью узнать сколько пользователей одновременно смогут авторизоваться в системе, были получены следующие результаты, изображенные на рисунке 13:

- Sample– количество отправленных запросов равно 500;
- Average– Среднее арифметическое для всех ответов (сумма всех раз /число) равно 41;
- Min (ms) –Минимальное время отклика (мс) равно 2 мс;
- Max (ms) – Максимальное время отклика (мс) равно 94 мс;
- Std Dev — Стандартное отклонение равно 32,62;
- Errorrate – процент неудачных тестов равно 0 процентов;
- Throughput – сколько запросов в секунду обрабатывает сервер равно 484,5 запроса в секунду;
- Received KB/Sec - количество полученных данных (кб/сек) равно 1709,46 кб/сек;
- Sent KB/Sec – количество отправленных данных (кб/сек) равно 78,07 кб/сек;
- Avg. Bytes – средний размер ответа равно 3613 байт.

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	500	41	2	94	32.62	0.00%	484.5/sec	1709.46	78.07	3613.0
TOTAL	500	41	2	94	32.62	0.00%	484.5/sec	1709.46	78.07	3613.0

Рисунок 13 – Результаты нагрузочного тестирования

Из результатов можно увидеть, что количество отправленных запросов равно 500, а процент ошибок составляет 0 процентов, из этого можно сделать вывод о том, что система выдерживает одновременную авторизацию 500 пользователей.

Теперь проведем ручное тестирование функциональных возможностей, но сначала нужно составить таблицу тестовых примеров, предполагаемого результата и полученного результата (Таблица 1).

Таблица 1 –Ручное тестирование информационной системы

Таблица, которая уместается на нескольких страницах, имеет особенности оформления, посмотрите в методичке. И еще, нельзя, чтобы

название таблицы отрывалось от самой таблицы, или чтобы в конце страницы был только заголовок таблицы и ее название.

№	Тестовый вариант	Ожидаемый результат	Результат теста
1	Открыть страницу авторизации.	Страница открыта, доступна и корректно отображается.	Пройден
2	Авторизация администратора или пользователя: корректная формулировка «На странице входа ввести верный логин и пароль» и «Нажать кнопку «Вход» (т. е. не «что делается», а «сделать»; поправьте, пожалуйста, тут и далее) 1. На странице входа вводятся верный логин и пароль. 2. Нажимается кнопка «Вход».	Вход произошел, происходит перенаправление на страницу с запланированными программами повышения квалификации.	Пройден
3	Авторизация администратора или пользователя с	Получение ошибки о том, что «неверный логин и/или пароль».	Пройден

	неправильным логином или паролем: 1. На странице входа вводятся неверный логин и/или пароль. 2. Нажимается кнопка «Вход».		
4	Редактирование информации о программе, на которую не записаны преподаватели: 1. На странице авторизации вводятся логин и пароль администратора. 2. Нажимается кнопка «Вход». 3. На открывшейся главной странице выбирается программа, на которую никто не записан. 4. После нажатия на программу будет выведена	Корректное отображение страниц и верные выводимые данные в таблице (на странице?) и базе.	Пройден

	подробная информация о ней. 5. Затем меняются любые данные, например, место проведения. 6. Нажимается кнопка «Обновить».		
5	Добавление новой программы: 1. На странице авторизации вводятся логин и пароль администратора. 2. Нажимается кнопка «Вход». 3. На открывшейся главной странице нажимается кнопка «Добавить ПК». 4. В открывшейся форме заполняем обязательные поля, т.е. название,	Корректное отображение страниц, правильное добавление в базу и верное отображение данных, взятых из БД, в таблице.	Пройден

	руководитель, дата начала, дата окончания. 5. Нажимается кнопка «Добавить». 6. При успешном добавлении выполнится перенаправление на главную страницу.		
6	Удаление программы повышения квалификации, на которую записаны преподаватели: 1. На странице авторизации вводятся логин и пароль администратора. 2. Нажимается кнопка «Войти». 3. На открывшейся главной странице выбирается программа, на которую	Корректное отображение страниц, получение сообщения об ошибке удаления программы (можно привести точную формулировку сообщения).	Пройден

	записаны преподаватели. 4. После нажатия на название программы будет выведена подробная информация о данной программе. 5. Нажимаем кнопку «Удалить».		
7	Регистрация преподавателя: 1. На странице авторизации вводятся обязательные данные, т.е. фамилия, имя, отчество, дата рождения, кафедра. 2. Нажимается кнопка «Регистрация». 3. Вывод сообщения об успешной	Корректное отображение данных и элементов страницы.	Пройден

	регистрации.		
--	--------------	--	--

Тестирование кроссбраузерности проводилось в разных браузерах: Google Chrome, Microsoft Edge, Яндекс.

В результате тестирования были обнаружены следующие ошибки.

Передача данных в формах редактирования из пустых полей приводило к ошибкам на сервере. Решение проблемы: проверка полей на пустоту, и если какое-то из полей было пустое, выводится сообщение о том, что какое-то из полей не заполнено или передача в базу данных пустого значения, **если оно допустимо в базе данных**.

На страницах редактирования после внесения изменений и нажатии кнопки «Изменить», **(не надо зпт)** не было никакого обновления, чтобы понять, что изменения были внесены. Решение проблемы: передача формам редактирования ссылки на страницу, **(не надо зпт)** со всеми записями о программах или информации о преподавателях, на которую происходит переадресация после внесения изменений.

После входа в систему, **(не надо зпт)** на странице входа можно было еще раз авторизоваться. Решение проблемы: проверка на вход в учетную запись, и если пользователь уже вошел, выводится сообщение «Вы уже вошли в учетную запись».

ЗАКЛЮЧЕНИЕ

В данной работе спроектирована и реализована информационная система «Учет повышения квалификации преподавателей вуза», которая позволяет сократить временные затраты работников кафедры или преподавателей и обеспечивает мгновенный доступ работникам и преподавателям к списку программ повышения квалификации. (точка) Система позволяет добавлять, хранить, редактировать и просматривать данные пользователя, у которого есть возможность авторизоваться в системе, и программ повышения квалификации. Про возможность записаться на программу ПК и заполнить учетную карточку слушателя у вас ничего тут не написано

В итоге была получена автоматизированная информационная система.

В процессе выполнения работы было сделано следующее:

- 1) Изучена предметная область.
- 2) Сформулированы задачи, подлежащие решению в рамках курсовой работы.
- 3) На основе анализа предметной области была построена информационная и логическая модели базы данных.
- 4) В СУБД MySQL реализована физическая модель базы данных.
- 5) Спроектировано клиентское приложение.
- 6) С использованием языка программирования Python и фреймворка Flask реализовано клиентское приложение для работы с созданной базой данных.
- 7) Реализована авторизация и регистрация пользователя, редактирование, добавление и просмотр данных о программах повышения квалификации и преподавателях.

8) С помощью созданной информационной системы были выполнены тестовые задачи по работе с повышениями квалификаций преподавателей.

9) Разработана инструкция пользователя по работе с созданной системой.

Таким образом, в ходе выполнения курсовой работы сформированы умения осуществлять постановку задачи и проводить оценку полученных результатов, осуществлять документирование разрабатываемого ПО, решать задачи по проектированию и реализации программного обеспечения в различных предметных областях, проводить испытания и оценку качества программного обеспечения, а также приобретены навыки использования инструментальных средств и языков для проектирования и производства программного обеспечения, выбора, проектирования и реализации программного обеспечения для решения задач в различных предметных областях, что свидетельствует о том, что компетенции ОПК-4, ОПК-9 и ОПК-11 освоены.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Разработка web-приложения «Повышение квалификации» для сайта кафедры ИС [Электронный ресурс] URL: <https://cyberleninka.ru/article/n/razrabotka-web-prilozheniya-povyshenie-kvalifikatsii-dlya-sayta-kafedry-is/viewer> (Дата обращения: 03.05.2022)
2. Кузнецов С.Д. Базы данных. — М. : Издательский центр «Академия», 2012. — 496 с.
3. Проектирование баз данных [Электронный ресурс] URL: <https://studizba.com/lectures/10-informatika-i-programmirovanie> (Дата обращения: 28.05.2022)
4. Что такое серверная веб-разработка – [Электронный ресурс] – URL: <https://igorosa.com/what-is-backend-development/> (Дата обращения: 15.04.2022)
5. Архитектура клиент-сервер: двухзвенная и трехзвенная– [Электронный ресурс] – URL: <https://www.freepapers.ru/28/serveryprilozhenij/260084.1722109.list1.html> (Дата обращения: 15.04.2022)
6. Руководство по MySQL – [Электронный ресурс] – URL: <https://metanit.com/sql/mysql/> (Дата обращения: 31.05.2022)
7. Flask – [Электронный ресурс] – URL: <https://help.sprinthost.ru/framework/flask> (Дата обращения: 03.02.2022)
8. Руководство по языку программирования Python – [Электронный ресурс] – URL: <https://metanit.com/python/tutorial/> (Дата обращения: 20.02.2022)
9. Jinja2 - движок шаблонов для Python – [Электронный ресурс] – URL: <https://docs-python.ru/packages/modul-jinja2-python/> (Дата обращения: 18.05.2022)

Логическая модель базы данных



ПРИЛОЖЕНИЕ Б

Листинги хранимых процедур

```
@app.route('/prog', methods=['GET', 'POST'])
def add_program():
    sql = "SELECT Id_teacher, Concat(Surname, ' ', Name, ' ', Patronymic)
FROM teacher"
    cur.execute(sql)
    direct = cur.fetchall()

    if request.method == "POST":
        title = request.form['title']
        director = request.form.get('director')
        date_start = request.form['date_start']
        date_end = request.form['date_end']
        holding_time = request.form['holding_time']
        venue = request.form['venue']
        teacher_program = request.form.get('teacher_program')

        try:
            sql = "SELECT Id_teacher FROM teacher WHERE Surname LIKE '%s'" %
director.split()[0]
            cur.execute(sql)
            id_direct = str(cur.fetchall()[0][0])
            sql = "SELECT Id_teacher FROM teacher WHERE Surname LIKE '%s'" %
teacher_program.split()[0]
            cur.execute(sql)
            id_teach = str(cur.fetchall()[0][0])
            cur.execute("INSERT INTO program_pd (Title, Director, Date_start,
Date_end, Holding_time, Venue, Teacher_program) VALUES (%s, %s, %s, %s, %s,
%s, %s)", (title, id_direct, date_start, date_end, holding_time, venue,
id_teach))
            myconn.commit()
            return redirect('/')
        except:
            return "При добавлении программы произошла ошибка."
    else:
        return render_template('prog.html', direct=direct)
```

Листинг Б.1 – добавление программы повышения квалификации

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    msg = ''
    if not session.get("email"):
        if request.method == 'POST' and 'email' in request.form and 'pass-
word' in request.form:
            email = request.form['email']
            password = request.form['password']
            cur.execute("SELECT * FROM user WHERE email=%s", (email))
            account = cur.fetchone()
            passw = account[2]
            hash = generate_password_hash(password)

            if account and check_password_hash(hash, passw):
                session['loggedin'] = True
                session['id'] = account[0]
```

```

        session['email'] = account[1]
        msg = 'Вы успешно вошли в систему !'
        return redirect(url_for('index'))
    else:
        msg = 'Некорректный логин / пароль !'
        return render_template('login.html', msg=msg)

elif session.get("email") == "admin@rt.ru":
    return render_template('mes_adm.html')
else:
    email = session["email"]
    cur.execute("SELECT Id_listener, Concat(Surname, ' ', Name, ' ',
Patronymic), Email FROM listener WHERE Email=%s",
                (email))
    listener = cur.fetchone()
    return render_template('mes_log.html', listener=listener)

```

Листинг Б.2 – функция входа в систему

```

<form action="" method="post">
  <fieldset>

    <label> Год прохождения ПК:</label>
    <input class="form-control" type="text" name="year_passage" place-
holder="Год прохождения ПК">

    <label for="Name">Образовательное учреждение:</label>
    <input list="list" name="educational_institution" class="form-con-
trol" id="Name" type="text" placeholder="Образовательное учреждение">
    <datalist id="list">
      {% for opt in instit %}
        <option value="{{ opt[1] }}">
      {% endfor %}
    </datalist>

    <label> Форма ПК:</label>
    <input class="form-control" type="text" name="form_pd" place-
holder="Форма ПК">

    <label> Наименование программы:</label>
    <input class="form-control" type="text" name="program_name" place-
holder="Наименование программы">

    <p> <label><button style="position: relative; left: 10px; top: 20px;"
formnovalidate type="submit" class="btn btn-warning">Добавить</button> <but-
ton style="position: relative; left: 10px; top: 20px;" type="reset"
class="btn btn-warning">Очистить</button></label></p>
  </fieldset>

  <br>
</form>

```

Листинг Б.3 - часть html шаблона добавления новой программы

```

{% extends 'base.html' %}

{% block title %}
Вход
{% endblock %}

{% block body %}

</br></br>
<div class="col-5" style="margin: 20px 80px">
  <div align="center" class="card mb-4 rounded-3 shadow-sm border-warn-
ing">
    <div align="center" class="card-header py-3 text-black bg-warning
border-warning">
      <h3 class="card-title pricing-card-title"> Вход </h3> <!--ФИО
преподавателя-слушателя-->
    </div>
    <div align="center" class="card-body">
      <ul align="center" class="list-unstyled mt-3 mb-4">
        <form action="{{ url_for('login') }}"method="post">
          <div class="msg">{{ msg }}</div>
          <input id="email"name="email"type="text"place-
holder="Введите логин"class="textbox"/></br></br>
          <input id="password"name="password"type="password"place-
holder="Введите пароль"class="textbox"/></br></br></br>
          <input type="submit" class="btn btn-warning"
value="Войти"></br></br>
        </form>
      </ul>
    </div>
  </div>
</div>
{% endblock %}

```

Листинг Б.4 - html шаблон входа в ситему

ПРИЛОЖЕНИЕ В

Клиентская часть системы

Учет ПК

Программы ПК

Информация о преподавателе

Информация о заявках на ПК

Записаться на ПК

ЛК

Вход

Регистрация

Добавить ПК

Список запланированных программ повышения квалификации

№ программы	Название	Руководитель	Дата начала	Дата окончания
1	Основы инновационной профессионально-ориентированной коммуникации на английском языке	Пыж Анна Михайловна	2021-03-16	2021-05-11
2	Стратегии делового и профессионального общения	Мишанина Мария Борисовна	2022-04-30	2022-05-30
3	Цифровой университет - цифровая педагогика	Соловова Наталья Валентиновна	2021-03-15	2021-05-15
4	Первичная подготовка по вопросам экспортного контроля	Мишанина Мария Борисовна	2022-04-23	2022-06-30
13	Воспитательная работа в университете	Сысоева Анастасия Юрьевна	2021-04-22	2021-05-27
14	Написание научных статей и подготовка презентаций на английском языке	Толстова Татьяна Викторовна	2021-03-11	2021-05-06
16	Проектирование и организация проектного обучения в рамках дисциплины	Свиридова Екатерина Дмитриевна	2021-03-17	2021-04-10
17	Облачные сервисы Google в реализации образовательных программ	Попова Маргарита Андреевна	2021-03-12	2021-04-30
18	Прикладная этика в профессиональной деятельности преподавателя высшей школы	Кузьмина Антонина Вячеславовна	2021-03-18	2021-05-20

Рисунок В.1 – Главная страница

Просмотр информации о преподавателе

Выберете преподавателя

ФИО преподавателя

ПОИСК

Найденные преподаватели:

Пушкин Александр Сергеевич

Рисунок В.2 - Информация о преподавателе

Добавление записи на программу повышения квалификации

Название программы:

Цифровой университет - цифровая педагогика

Слушатель программы:

Саблина Екатерина Матвеевна

Записаться

Очистить

Рисунок В.3 – Запись на программу повышения квалификации

The screenshot shows a login interface with a yellow header containing the word "Вход" (Login). Below the header, there are two white input fields with gray borders. The first field contains the placeholder text "Введите логин" (Enter login), and the second field contains "Введите пароль" (Enter password). Below these fields is a yellow button with the text "Войти" (Login).

Рисунок В.4 – Страница авторизации (пустую не надо показывать, хотя бы логин-пароль туда воткните)

The screenshot displays a web page with a dark gray navigation bar at the top. The bar contains several links: "Учет ПК" (underlined), "Программы ПК", "Информация о преподавателе", "Информация о заявках на ПК", and a yellow button labeled "Записаться на ПК". Below the navigation bar is a yellow header with the text "Основы инновационной профессионально-ориентированной коммуникации на английском языке". The main content area is white and contains the following text:
Руководитель: Пыж Анна Михайловна
Дата начала: 2021-03-16
Дата окончания: 2021-05-11
Время проведения: По вторникам в 15:15
Место проведения: Корп. 27, ауд.302 (ул. Потапова, д. 64/163)
Преподаватель программы: Соловова Наталья Валентиновна

Рисунок В.5 – Подробная информация о программе повышения квалификации

Пушкин Александр Сергеевич

Кафедра: Кафедра русского языка и массовой коммуникации

Должность: профессор

Ученая степень: Доктор наук

Ученое звание: Профессор

Стаж: 32 лет

Дата рождения: 1964-03-12

Телефон: 89999999999

Email: pushkin@mail.ru

Редактировать

Прошедшие ПК

№ записи ПК	Год прохождения	Образовательное учреждение	Форма ПК	Наименование программы
18	2017	Самарский университет	Курсы ФПК	Какое-нибудь название
19	2021	АНО ДПО "Корпоративный университет Сбербанка"	Курсы ФПК	Какое-нибудь название

*Для просмотра прошедшей ПК наведите на соответствующую строку в таблице.

Добавить ПК

Рисунок В.6 – Подробная информация о преподавателе

Редактирование информации о преподавателе: Пушкин Александр Сергеевич

Фамилия:

Пушкин

Имя:

Александр

Отчество:

Сергеевич

Название кафедры:

Кафедра русского языка и массовой коммуникации

Должность:

профессор

Ученая степень:

Доктор наук

Ученое звание:

Профессор

Стаж:

32

Дата рождения:

12.03.1964



Телефон:

89999999999

Обновить

Рисунок В.7 – Редактирование личной информации о преподавателе

Редактирование программы повышения квалификации

Название программы:

Основы инновационной профессионально-ориентированной коммуникации на английском языке

Руководитель программы

1

Дата начала курса:

16.03.2021

Дата окончания курса:

11.05.2021

Время проведения:

По вторникам в 15:15

Место проведения:

Корп. 27, ауд.302 (ул. Потапова, д. 64/163)

Преподаватель программы

3

Обновить

Очистить

Рисунок В.8 – Редактирование информации о программе повышения квалификации

ПРИЛОЖЕНИЕ Г

Данная инструкция пользователя предназначена для того, чтобы объяснить, как добавлять новые записи в базу данных.

Рассмотрим добавление записи «прошедший курс повышения квалификации». Нужно зайти в личный кабинет и в форме «Прошедшие курсы повышения квалификации» нажать на кнопку «Добавить ПК». На странице добавления нужно заполнить все поля или выбрать из выпадающих списков нужные критерии. После заполнения всех полей, (не надо зпт) надо нажать кнопку «Добавить».

В результате проделанных действий на главной странице веб-приложения появится запись, содержащая информацию о созданной программе.

Для записи на курс повышения квалификации необходимо нажать в шапке на кнопку «Записаться на ПК». Далее надо выбрать подходящую программу повышения квалификации и нажать кнопку «Записаться».

При редактировании записей в форме редактирования перед тем, как применить изменения, обязательно нужно убедиться, что все поля заполнены, (а тут надо зпт) и выбраны критерии из выпадающих списков.